

IF1 Forall Lowering Algorithm

The lowering of Sisal **for-all** loops to the IF1 intermediate representation follows a structured approach that decouples the iteration space (**Generator**) from the computation (**Body**) and the result aggregation (**Returns**). This design, aligned with Sisal 2.0, ensures that multidimensional loops and array placements are handled orthogonally.

1. The FORALL Compound Structure

A Sisal **forall** expression is represented by a single **FORALL** compound node containing three primary subgraphs: 1. **GENERATOR**: Enumerates the index space and produces loop variables (indices and elements). 2. **BODY**: Computes the per-iteration values. It is “flat” and executes once per iteration point defined by the generator. 3. **RETURNS**: Gathers the values produced by the body into the final results (arrays, streams, or scalars).

2. Lowering Algorithm

Step 1: Generator Analysis and Axis Flattening

- **Flattening**: Analyze the **in** expressions. **dot** (zip) axes are grouped into a single iteration level, while **cross** (Cartesian product) axes define a nested iteration space.
- **Axis Classification**:
 - **Range Axes** (**i in lo..hi**): Lowered to **RANGEGEN** nodes. These produce the induction variable, lower bound, and upper bound.
 - **Scatter Axes** (**x in A**): Lowered to **DV_SCATTER** (or **ASCATTER**) nodes. These produce the array elements and optional **at** indices.

Step 2: Construct the GENERATOR Subgraph

- For each axis (or group of **dot** axes):
 - Emit the corresponding generator nodes (**RANGEGEN**, **DV_SCATTER**).
 - **Recursive Nesting**: For **cross** loops, the generator subgraphs are nested. The outermost axis forms the top-level generator; inner axes are contained within nested **FORALL** generators or sub-generator structures.
- **Exporting Symbols**: Every loop variable (index **i**, element **x**) and range bound (**lb**, **ub**) is added to the generator’s **boundary outputs**. These values are “published” to the parent **FORALL** scope.

Step 3: Construct the BODY Subgraph

- **Lazy Import**: The body subgraph imports only the names it references (outer scope variables or generator-produced loop variables) by creating **boundary inputs**.

- **Lowering:** The Sisal body expressions are lowered to standard IF1 nodes (arithmetic, function calls, etc.).
- **Handoff:** Each expression destined for a `returns` clause is wired to a **boundary output** of the body subgraph.

Step 4: Construct the RETURNS Subgraph

- **Gathering:** For each result in the `returns` clause, a gather node is emitted:
 - `array of:` DV_GATHER (or AGATHER).
 - `stream of:` SGATHER.
 - Reductions: REDUCE (e.g., SUM, PRODUCT).
- **Decoupled Placement (at):**
 - The `index` port of the gather node is wired to the iteration coordinate. If an explicit `at` clause is present, the placement expression is used instead.
 - The `value` port is wired to the corresponding body output.
- **Multidimensional Results:** If a `cross` loop builds a multidimensional array, the RETURNS subgraph mirrors the generator's nesting. Each level of the nested gather assembly handles one dimension of the result.

Step 5: Final Wiring and Normalization

- **Compound Wiring:** Connect the FORALL compound inputs to the required outer-scope symbols.
- **Lower Bound Adjustment:** If the result is a 1-D array from a range loop, an ASETL node is appended to the FORALL output to ensure the array's lower bound matches the loop's specification (defaulting to 1).
- **Type Consistency:** Ensure the FORALL node's output ports match the types and counts of the RETURNS subgraph's results.

3. Summary of IF1 Nodes

Node	Function
FORALL	Compound node encapsulating the loop logic.
RANGEGEN	Produces induction variables and bounds for range loops.
DV_SCATTER	Scatters a flat array (Dope Vector) into elements.
DV_GATHER	Assembles elements into a flat array (Dope Vector).
REDUCE	Performs associative reductions (Sum, Max, etc.).
ASETL	Sets the lower limit of an array result.

4. Muchnick-style Pseudo-Pascal Implementation

```
procedure LowerForAll(ForExp: AstNode)
  input: ForExp - a Sisal AST node representing "for ... in ... returns ... end for"
  output: ForAllNode - an IF1 FORALL compound node

  var
    GenGraph, BodyGraph, RetGraph: Graph;
    Axes: list of Axis;
    OutPorts: list of Port;

  begin
    { Step 1: Analyze the iteration space }
    Axes := FlattenAxes(ForExp.InExp); { Separate Dot from Cross }

    { Step 2: Construct the recursive Generator }
    GenGraph := CreateNewGraph(ParentScope);
    foreach Axis in Axes do
      if Axis.IsRange then
        AddNode(GenGraph, RANGEGEN, Axis.Lo, Axis.Hi);
      else if Axis.IsScatter then
        AddNode(GenGraph, DV_SCATTER, Axis.SourceArray);
      end if;
      RegisterLoopVariables(GenGraph.SymTab, Axis);
    end foreach;
    PublishGeneratorOutputs(GenGraph);

    { Step 3: Construct the Body (Decoupled and Flat) }
    BodyGraph := CreateNewGraph(GenGraph);
    LowerExpression(BodyGraph, ForExp.BodyExp);
    foreach ReturnClause in ForExp.Returns do
      BodyPort := ExportToBoundary(BodyGraph, ReturnClause.Value);
      Add(OutPorts, BodyPort);
    end foreach;

    { Step 4: Construct the Returns Graph (Gathering) }
    RetGraph := CreateNewGraph(GenGraph);
    if IsMultidimensional(Axes) then
      ConstructNestedGathers(RetGraph, Axes, ForExp.Returns);
    else
      foreach ReturnClause in ForExp.Returns do
        GatherNode := CreateGatherNode(RetGraph, ReturnClause.Kind);
        WireIndex(GatherNode, GetPlacement(ReturnClause));
        WireValue(GatherNode, ImportFromBody(ReturnClause));
      end foreach;
    end if;
  end;
```

```

{ Step 5: Assemble the FORALL Compound }
ForAllNode := CreateCompound(If1_FORALL);
InstallSubgraph(ForAllNode, GenGraph, "GENERATOR");
InstallSubgraph(ForAllNode, BodyGraph, "BODY");
InstallSubgraph(ForAllNode, RetGraph, "RETURNS");

{ Step 6: Finalize Wiring and Normalization }
WireOuterInputs(ForAllNode);
foreach Port in ForAllNode.Outputs do
    if NeedsLowerBoundAdjustment(Port) then
        InsertASETl(Port, GetLowerBound(Axes));
    end if;
end foreach;

return ForAllNode;
end procedure

```

Key Procedural Components:

- **FlattenAxes:** Decomposes the Sisal **in** expression into a list of axes, distinguishing between **dot** (combined into one level) and **cross** (forming a hierarchy).
- **PublishGeneratorOutputs:** Scans the generator's local symbol table and creates boundary outputs for induction variables, elements, and range bounds so they are visible to the **BODY** and **RETURNS** subgraphs.
- **ConstructNestedGathers:** A recursive procedure that builds the **RETURNS** nest. For a rank- N result, it nests N gather nodes, where each level handles one dimension of the **cross** product.
- **ImportFromBody:** Connects a **RETURNS** input to a named **BODY** output via a dataflow edge in the **FORALL** compound scope.
- **InsertASETl:** Ensures that the resulting **array_dv** has the correct starting index (lower bound) by appending an **ASETl** node to the **FORALL** output.

5. Detailed Component Descriptions

The following procedures implement the core logic of the lowering phase, ensuring that the decoupling of scatter and gather is maintained while preserving dataflow integrity.

5.1 Iteration Space Analysis

- **FlattenAxes(InExp):** Performs a structural analysis of the Sisal **in** expression. It deconstructs nested **dot** and **cross** AST nodes into a canonical list of axis descriptors.
 - **Dot Axes:** Grouped together; they operate in lockstep at the same nesting level.

- **Cross Axes**: Ordered outermost-to-innermost, defining the recursive depth of the generator.
- **GetLowerBound(Axes)**: Determines the starting index for the result array. For range loops, it evaluates the expression provided in the **lo** part of the triplet. For element loops, it queries the dope vector of the source array at runtime.

5.2 Symbol Management and Graph Construction

- **CreateNewGraph(Scope)**: Instantiates a new IF1 subgraph. It initializes a local symbol table that inherits from the **Scope**’s parent table, establishing the lexical scoping rules required for loop-invariant access.
- **RegisterLoopVariables(SymTab, Axis)**: Inserts the names produced by the generator (e.g., indices, elements) into the local symbol table. Each entry is tagged with the node and port index of the **RANGE**GEN or **DV_SCATTER** node that defines it.
- **PublishGeneratorOutputs(GenGraph)**: Scans the generator’s local symbol table for newly defined names. For each such name, it:
 1. Allocates a unique **Boundary Output** port on the **GenGraph**.
 2. Lays a dataflow edge from the internal defining node to this new boundary port. This makes the loop variables available to the **BODY** and **RETURNS** subgraphs via the parent **FORALL** compound.

5.3 Body and Dataflow Integration

- **LowerExpression(BodyGraph, Expr)**: Recursively lowers the body of the loop. It treats the body as a “black box” computation that consumes loop variables (imported via boundary inputs) and produces values for the returns clause.
- **ExportToBoundary(Graph, Value)**: Identifies the terminal node in a computation and wires its output to a boundary port. This creates a “handle” that sibling subgraphs can reference.
- **ImportFromBody(ReturnClause)**: Resolves a name reference in the **RETURNS** subgraph by searching the parent **FORALL** symbol table. It identifies the corresponding output port on the **BODY** compound and wires it to a boundary input of the **RETURNS** subgraph.

5.4 Gather and Result Assembly

- **ConstructNestedGathers(RetGraph, Axes, Returns)**: The recursive heart of multidimensional array building.
 - For a rank- N result, it generates N levels of nested gathers.
 - Each level consumes one axis’s bounds (to size the dimension) and one axis’s induction variable (to determine placement).
 - The innermost level consumes the body’s value.
- **CreateGatherNode(RetGraph, Kind)**: Emits the specific IF1 gather primitive:

- DV_GATHER: For flat, dope-vector arrays.
- SGATHER: For stream production.
- REDUCE: For scalar aggregations (e.g., SUM).
- **WireIndex(GatherNode, Placement)**: Connects the `index` port of a gather node. This is where the Sisal 2.0 `at` expression lands, allowing for arbitrary permutations and non-contiguous writes.

5.5 Normalization

- **InsertASETl(Port, LowerBound)**: A post-processing step for array results. Since IF1 gather nodes typically produce arrays starting at index 0 or 1 by default, **ASETl** (Array Set Lower Limit) is used to stamp the user-specified lower bound onto the final result before it leaves the **FORALL** compound.